Studio Nx

P1-05 로그인 API — 함수 설명 합본 (26~34)

2026-10-12

목차

1. ai-notes/backend(fastAPI_Python)/P1-05 - 로그인 API/26_security_create_access_token.md
2. ai-notes/backend(fastAPI_Python)/P1-05 - 로그인 API/27_security_create_refresh_token.md
3. ai-notes/backend(fastAPI_Python)/P1-05 - 로그인 API/28_security_decode_refresh_token.md
4. ai-notes/backend(fastAPI_Python)/P1-05 - 로그인 API/29_auth_service_authenticate.md
5. ai-notes/backend(fastAPI_Python)/P1-05 - 로그인 API/30_auth_service_get_active_user.md
6. ai-notes/backend(fastAPI_Python)/P1-05 - 로그인 API/31_auth_route_refresh_cookie.md
7. ai-notes/backend(fastAPI_Python)/P1-05 - 로그인 API/32_auth_route_login.md
8. ai-notes/backend(fastAPI_Python)/P1-05 - 로그인 API/33_auth_route_refresh.md
9. ai-notes/backend(fastAPI_Python)/P1-05 - 로그인 API/34_auth_route_logout.md

Backend(FastAPI/Python) 26 — `create_access_token()` · `_create_token()`

항목	내용
파일	`studionx/backend/app/core/security.py`
작업	P1-05 로그인: Access Token 발급 (PRD 01 FR-103, FR-105)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	25 입력 스키마 (P1-04 끝) · 27 `create_refresh_token()`

1. 하는 일

회원 번호로 **30분짜리 Access Token(JWT)**을 만듭니다. 05번 `decode_access_token()`이 읽는 토큰을 만드는 쪽입니다.

`create_access_token(7)` → `"eyJhbGciOiJIUzI1NiIs….eyJzdWIiOiI3Iiwi….kQ3x…"`

토큰 안의 칸	값	뜻
`sub`	`"7"`	토큰의 주인입니다 (회원 번호, 문자열)
`type`	`"access"`	토큰의 종류로, 리프레시 토큰(27)과 구분합니다
`iat`	`1791339232`	만든 시각입니다 (1970년부터 센 초)
`exp`	`1791341032`	만료 시각으로, `iat` + 1800초(30분)입니다

함수	역할
`create_access_token()`	바깥에서 부르는 이름입니다. 수명 30분을 정합니다
`_create_token()`	실제로 토큰을 만드는 곳입니다. 27번 `create_refresh_token()`도 같은 함수를 씁니다 (05번의 `_decode_token`과 짝을 이룹니다)

2. 언제 쓰나

쓰는 곳	화면	하는 일
P1-05 `POST /auth/login` (32)	home.html 로그인 모달	이메일과 비밀번호가 맞으면 응답 `{"access_token": …}`으로 줍니다
P1-05 `POST /auth/refresh` (33)	모든 페이지 (`api.js` 01)	리프레시 쿠키가 유효하면 새 Access Token을 줍니다

받은 토큰은 브라우저 메모리에 두고(`setAccessToken`), 모든 API 요청의 `Authorization: Bearer …`에 붙입니다 (JS 02). 30분이 지나면 서버(05)가 401로 응답하고, 브라우저는 33번으로 새 토큰을 받습니다.

3. 코드

import (파일 맨 위)

```
import re
import string
from datetime import UTC, datetime, timedelta

import bcrypt
import jwt

from app.core.config import get_settings
from app.core.errors import AppError, ErrorCode
```

`security.py` 맨 위의 import입니다. **`from datetime import UTC, datetime, timedelta`를 이번에 추가했습니다.** 이 함수들은 이것과 `jwt`, `get_settings`를 사용합니다.

import	쓰는 곳
`UTC`, `datetime`, `timedelta`	지금 시각(UTC)과 30분이라는 기간
`jwt`	`jwt.encode`로 토큰을 만듭니다 (05번의 `jwt.decode`와 반대 방향)
`get_settings`	`.env`의 `JWT_SECRET`과 `jwt_access_minutes`(30)

함수

```
JWT_ALGORITHM = "HS256"


def create_access_token(user_id: int) -> str:
    minutes = get_settings().jwt_access_minutes
    return _create_token(user_id, "access", timedelta(minutes=minutes))


def _create_token(user_id: int, token_type: str, lifetime: timedelta) -> str:
    now = datetime.now(UTC)
    payload = {
        "sub": str(user_id),
        "type": token_type,
        "iat": now,
        "exp": now + lifetime,
    }
    return jwt.encode(payload, get_settings().jwt_secret, algorithm=JWT_ALGORITHM)
```

문법 풀이

import 문법	뜻
`from datetime import UTC, datetime, timedelta`	파이썬 기본 `datetime` 모듈에서 세 이름을 가져옵니다

식	뜻
`timedelta(minutes=30)`	30분이라는 **기간**입니다
`datetime.now(UTC)`	지금 시각입니다 (UTC, 시간대 표시가 있습니다)
`now + lifetime`	30분 뒤의 시각입니다
`jwt.encode(payload, 비밀값, algorithm="HS256")`	머리, 내용, 서명 세 부분을 만들어 `.`으로 이은 문자열입니다. `datetime` 값은 PyJWT가 초 단위 숫자로 바꿔 넣습니다
`str(user_id)`	`7`을 `"7"`로 바꿉니다. JWT 규격에서 `sub`는 문자열이기 때문입니다

식	뜻
`_create_token`의 `_`	이 파일 안에서만 쓰는 함수라는 표시입니다 (05번의 `_decode_token`과 같은 관례)

4. 흐름

1. 설정에서 수명(30분)을 읽습니다.
2. 지금 시각(UTC)을 잽니다.
3. `sub`, `type`, `iat`, `exp`를 담습니다.
4. 서버의 비밀값으로 서명해서 토큰 문자열을 돌려줍니다.

5. 왜 이렇게

선택	이유
수명 30분	이 토큰은 가진 사람을 인정하는 방식(Bearer)이라서, 누가 훔쳐 가더라도 오래 쓰지 못하게 짧게 둡니다. 대신 리프레시(27, 33)를 이용해서 사용자는 계속 로그인된 것처럼 쓸 수 있습니다
수명은 `.env` 설정에서 읽음	코드를 고치지 않고도 바꿀 수 있습니다 (`jwt_access_minutes`)
`type`을 넣음	05번이 `type`을 검사해서, 3일짜리 리프레시 토큰을 Access 토큰 자리에 쓰지 못하게 막습니다
`iat`(만든 시각)를 추가함 (템플릿에는 없음)	같은 회원이 같은 초에 두 번 받지 않는 한 토큰이 매번 달라집니다. 또 나중에 "이 시각 이전의 토큰은 무효" 같은 처리를 할 수 있습니다. 05번은 이 칸을 요구하지 않으므로 영향이 없습니다
만드는 일을 `_create_token`으로 분리	27번 `create_refresh_token`은 수명과 종류만 다르므로 한 줄로 끝납니다
알고리즘은 05번과 같은 `JWT_ALGORITHM`을 씀	만드는 쪽과 읽는 쪽이 같은 상수를 씁니다

토큰 내용은 암호화되지 않습니다 (05 §2). 누구나 읽을 수 있으므로, 회원 번호 외에 이메일이나 권한 같은 정보는 넣지 않습니다.

6. 시험 (`tests/test_security.py`에 5개를 추가해서 모두 14개)

시험	확인한 것
만들고 읽기	`decode_access_token(create_access_token(7)) == 7`입니다. 05번과 짝이 맞습니다
내용	`sub`는 `"7"`, `type`은 `"access"`, `exp - iat`는 1800초입니다
머리	`alg`는 `HS256`입니다
시각	`iat`가 지금 시각과 2초 안으로 차이 나고, `exp`는 `iat` + 1800입니다
회원마다 다름	7번 회원과 8번 회원의 토큰이 다릅니다

05번 시험에서 쓰던 `jwt.encode` 직접 만들기(`make_token`)는 그대로 둡니다. 만료되었거나 위조된 이상한 토큰은 이 함수로 만들 수 없기 때문입니다.

전체 시험 191개가 통과했고, `ruff` 검사도 통과했습니다.

Backend(FastAPI/Python) 27 — `create_refresh_token()`

항목	내용
파일	`studionx/backend/app/core/security.py`
작업	P1-05 로그인: Refresh Token 발급 (PRD 01 FR-105, FR-109)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	26 `create_access_token()` · 28 `decode_refresh_token()`

1. 하는 일

회원 번호로 **3일짜리 Refresh Token(JWT)**을 만듭니다. 26번과 모양이 같고, 종류와 수명만 다릅니다.

	Access Token (26)	Refresh Token (이 함수)
`type`	`"access"`	`"refresh"`
수명	30분	**3일** (`jwt_refresh_days`)
브라우저에 두는 곳	JS 메모리 (`api.js` 변수)	**HttpOnly 쿠키 `snx_refresh`**입니다. JS가 읽을 수 없습니다
서버로 가는 경로	모든 API 요청의 `Authorization: Bearer …`	`/api/v1/auth/…` 요청에만 쿠키로 자동으로 실립니다
쓰임	API 요청	새 Access Token을 받을 때 씁니다 (33 `/auth/refresh`)

2. 언제 쓰나

쓰는 곳	하는 일
P1-05 `POST /auth/login` (32)	로그인에 성공하면 이 토큰을 쿠키에 담아 응답합니다 (31 `set_refresh_cookie`)

두 토큰이 함께 움직이는 순서

순서	일어나는 일
①	로그인하면 Access(30분)는 응답 본문으로, Refresh(3일)는 쿠키로 받습니다
②	30분 동안의 API 요청은 Access로 보냅니다
③	Access가 만료되면 서버가 401로 응답하고, 브라우저가 `POST /auth/refresh`를 보냅니다 (쿠키가 자동으로 실립니다)
④	서버가 Refresh를 검사하고(28) 새 Access를 만들어 줍니다(26)
⑤	3일이 지나면 Refresh도 만료되어 다시 로그인해야 합니다

자동 로그인 체크와의 관계: 토큰의 수명은 **항상 3일**입니다. 체크 여부는 **쿠키**의 수명만 바꿉니다 (31).

로그인 모달의 "Keep me signed in"	쿠키	결과
체크함	3일짜리 쿠키	브라우저를 닫았다가 다시 열어도 3일 동안 로그인이 유지됩니다
체크하지 않음	세션 쿠키 (수명 없음)	브라우저를 닫으면 쿠키가 사라져서 다시 로그인해야 합니다

3. 코드

import (파일 맨 위)

```
import re
import string
from datetime import UTC, datetime, timedelta

import bcrypt
import jwt

from app.core.config import get_settings
from app.core.errors import AppError, ErrorCode
```

`security.py` 맨 위의 import입니다 (26번과 같고, 이번에 새로 추가한 것은 없습니다). 이 함수는 **`timedelta`와 `get_settings`**를 쓰고, 토큰을 만드는 일은 26번의 `_create_token()`에 맡깁니다.

함수

```
def create_refresh_token(user_id: int) -> str:
    days = get_settings().jwt_refresh_days
    return _create_token(user_id, "refresh", timedelta(days=days))
```

문법 풀이

import 문법	뜻
`from datetime import timedelta`	기간을 나타내는 타입입니다 (26번에서 이미 가져왔습니다)

식	뜻
`get_settings().jwt_refresh_days`	`.env` 설정입니다. 기본값은 3입니다 (`core/config.py`)
`timedelta(days=3)`	3일, 즉 259,200초입니다
`_create_token(user_id, "refresh", …)`	26번과 같은 함수에 종류와 수명만 다르게 넘깁니다

4. 흐름

1. 설정에서 수명(3일)을 읽습니다.
2. 26번의 `_create_token()`으로 `type: "refresh"` 토큰을 만듭니다.

5. 왜 이렇게

선택	이유
Access 토큰과 나눔	자주 오가는 Access는 짧게(30분) 두고, 오래 가는 Refresh는 JS가 읽을 수 없는 쿠키에 숨깁니다. 스크립트 공격(XSS)으로 Access를 훔치더라도 30분이 지나면 쓸 수 없습니다
`type: "refresh"`	05번은 Access 자리에서 이 토큰을 거절하고, 28번은 반대로 Access 토큰을 거절합니다. 그래서 두 토큰을 서로 바꿔 쓸 수 없습니다
토큰 수명은 체크 여부와 상관없이 3일	쿠키가 브라우저에서 사라지면 토큰도 쓸 일이 없습니다. 그래서 토큰을 두 가지로 만들 필요가 없습니다

선택	이유
같은 비밀값과 알고리즘	26번과 같은 `_create_token`을 써서 서명 방식이 하나로 유지됩니다

6. 시험 (`tests/test_security.py`에 3개를 추가해서 모두 17개)

시험	확인한 것
내용	`sub`는 `"7"`, `type`은 `"refresh"`, `exp - iat`는 259,200초(3일)입니다
Access 자리에 쓰기	05번 `decode_access_token`이 401로 응답합니다. 3일짜리 토큰으로 API를 부를 수 없습니다
두 토큰은 다름	같은 회원의 Access와 Refresh가 서로 다릅니다

전체 시험 194개가 통과했고, `ruff` 검사도 통과했습니다.

Backend(FastAPI/Python) 28 — `decode_refresh_token()`

항목	내용
파일	`studionx/backend/app/core/security.py`
작업	P1-05 로그인: Refresh Token 검사 (PRD 01 FR-105)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	27 `create_refresh_token()` · 29 `authenticate()`

1. 하는 일

리프레시 쿠키에 담긴 토큰이 **진짜이고 아직 유효한 Refresh Token**인지 확인하고, 회원 번호를 돌려줍니다. 실패하면 401로 응답합니다.

`decode_refresh_token("eyJ…")` → `7`

05번 `decode_access_token()`과 같은 검사이며, `type`만 `"refresh"`여야 한다는 점이 다릅니다.

함수	받아들이는 토큰
05 `decode_access_token()`	`type: "access"`만 받습니다
28 `decode_refresh_token()`	`type: "refresh"`만 받습니다

2. 언제 쓰나

쓰는 곳	화면	하는 일
P1-05 `POST /auth/refresh` (33)	모든 페이지의 `api.js` `refreshAccessToken()` (JS 01)	쿠키 `snx_refresh`의 토큰을 이 함수로 검사하고, 회원을 확인한 뒤 새 Access Token을 줍니다

쿠키의 토큰	결과	브라우저
정상 (3일 안)	`7`을 돌려주고 새 Access Token을 줍니다	로그인이 유지됩니다
만료, 위조, 종류가 `access`, 엉터리 문자열	401 "Please sign in again."	`refreshAccessToken()`이 `false`를 돌려주고 로그인 화면으로 갑니다

3. 코드

import (파일 맨 위)

```
import re
import string
from datetime import UTC, datetime, timedelta

import bcrypt
import jwt
```

```
from app.core.config import get_settings
from app.core.errors import AppError, ErrorCode
```

`security.py` 맨 위의 import입니다 (이번에 새로 추가한 것은 없습니다). **이 함수는 import한 것을 직접 쓰지 않습니다.** 검사는 같은 파일의 05번 `_decode_token()`이 하고, 그 함수가 `jwt`, `get_settings`, `AppError`를 사용합니다.

함수

```
def decode_refresh_token(token: str) -> int:
    return _decode_token(token, "refresh")
```

05번에서 검사 부분을 `_decode_token(token, expected_type)`으로 나눠 둔 덕분에 한 줄로 끝납니다. 05번 문서 §5에 "P1-05의 `decode_refresh_token()`은 `return _decode_token(token, "refresh")` 한 줄이 된다"고 적어 둔 그대로입니다.

문법 풀이

import 문법	뜻
(없음)	같은 파일의 함수는 import 없이 부릅니다

`_decode_token(token, "refresh")`가 하는 검사는 다음과 같습니다 (05번과 같음).

순서	검사	실패하면
①	서명, 만료, 필수 칸(`exp`, `sub`, `type`)을 확인하고, 알고리즘은 `HS256`만 허용합니다	401
②	`type`이 `"refresh"`인지 확인합니다	401
③	`sub`를 정수로 바꿉니다	401

4. 흐름

1. 05번의 `_decode_token()`에 종류 `"refresh"`를 넘깁니다.
2. 통과하면 회원 번호를 돌려주고, 아니면 401로 응답합니다.

5. 왜 이렇게

선택	이유
05번과 같은 검사 함수를 씀	서명, 만료, 알고리즘 고정(`alg: none` 차단) 규칙이 한곳에만 있게 됩니다. 05번을 고치면 이 함수도 함께 바뀝니다
Access Token을 거절함	30분짜리 토큰으로 재발급을 받을 수 있다면, 훔친 Access Token으로 새 토큰을 계속 받아서 30분 제한을 무력화할 수 있습니다
실패 이유와 상관없이 같은 401	05번과 같은 이유입니다. 브라우저는 어떤 실패든 "다시 로그인" 하나로 처리합니다
회원 상태(탈퇴, 정지)는 여기서 보지 않음	이 함수는 토큰만 봅니다. 회원 확인은 33번 라우터가 30번 `get_active_user()`로 합니다 (06번 `get_current_user`와 같은 역할 분담)

6. 시험 (`tests/test_security.py`에 8개를 추가해서 모두 25개)

시험	확인한 것
만들고 읽기	`decode_refresh_token(create_refresh_token(7)) == 7`입니다 (27번과 짝)
Access Token을 넣음	401입니다. 26번으로 만든 토큰은 거절합니다
이상한 토큰 6개	만료, 다른 비밀값, `sub: "admin"`, `exp` 없음, 엉터리 문자열, 빈 문자열은 모두 401 "Please sign in again."입니다

전체 시험 202개가 통과했고, `ruff` 검사도 통과했습니다.

Backend(FastAPI/Python) 29 — `authenticate()` • `invalid_credentials()`

항목	내용
파일	`studionx/backend/app/services/auth_service.py`
작업	P1-05 로그인: 이메일과 비밀번호 확인 (PRD 01 FR-103, FR-108)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	28 `decode_refresh_token()` · 30 `get_active_user()`

1. 하는 일

로그인 모달에 입력한 이메일과 비밀번호가 맞는지 확인합니다. 맞으면 회원을 돌려주고 **마지막 로그인 시각**을 기록합니다.

경우	결과	로그인 모달 문구
이메일과 비밀번호가 맞고, 활동 회원임	`User`를 돌려주고 `last_login_at`을 지금으로 기록합니다	(로그인됩니다)
비밀번호가 틀림	401 `INVALID_CREDENTIALS`	Email or password is incorrect.
없는 이메일	401 `INVALID_CREDENTIALS`이며, 위와 **문구도 같고 걸리는 시간도 같습니다**	Email or password is incorrect.
탈퇴한 회원 (비밀번호가 맞아도)	401 `INVALID_CREDENTIALS`이며, 위와 같습니다	Email or password is incorrect.
정지된 회원이고 비밀번호가 **맞음**	403 `ACCOUNT_DISABLED`	This account is disabled.
정지된 회원이고 비밀번호가 틀림	401 `INVALID_CREDENTIALS`	Email or password is incorrect.

함수	역할
`authenticate()`	확인하고 기록합니다
`invalid_credentials()`	401 오류를 만듭니다 (네 곳에서 같은 오류를 씁니다)

2. 언제 쓰나

쓰는 곳	화면
P1-05 `POST /auth/login` (32)	home.html 로그인 모달의 **Sign In** 버튼

32번이 이 함수로 회원을 확인한 뒤, 26번과 27번으로 토큰 두 개를 만들어 줍니다.

3. 코드

import (파일 맨 위)

```
from datetime import UTC, datetime, timedelta
```

```
from sqlalchemy import select
from sqlalchemy.exc import IntegrityError
from sqlalchemy.orm import Session

from app.core.errors import AppError, ErrorCode
from app.core.security import check_password_rules, hash_password, verify_password
from app.models import User
from app.models.user import UserStatus
from app.schemas.auth import PRIVACY_VERSION, AvailabilityResponse, SignupRequest
```

`auth_service.py` 맨 위의 import입니다. **`verify_password`를 이번에 추가했습니다.** 이 함수는 `datetime`, `UTC`, `select`, `Session`, `AppError`, `ErrorCode`, `verify_password`, `User`, `UserStatus`를 사용합니다.

함수

```
# 없는 이메일로 로그인할 때 비교에 쓰는 가짜 해시 (bcrypt 비용 12 — 진짜 해시와 같은 시간이 걸림).
# 어떤 비밀번호와도 맞지 않는다. 비밀값이 아니므로 코드에 두어도 됨.
DUMMY_PASSWORD_HASH = "$2b$12$1YCmgxF3pcmPkm7MWpsgVuOD.w6LlO7o43SuOZLbLZkz.glHDfmva"


def authenticate(db: Session, email: str, password: str) -> User:
    user = db.scalar(select(User).where(User.email == normalize_email(email)))
    if user is None:
        verify_password(password, DUMMY_PASSWORD_HASH)  # 없는 이메일도 같은 시간(약 0.2초)이 걸리게
        raise invalid_credentials()
    if not verify_password(password, user.password_hash):
        raise invalid_credentials()
    if user.status == UserStatus.DELETED:
        raise invalid_credentials()
    if user.status == UserStatus.SUSPENDED:
        raise AppError(ErrorCode.ACCOUNT_DISABLED, 403, "This account is disabled.")
    user.last_login_at = datetime.now(UTC).replace(tzinfo=None)
    db.commit()
    return user


def invalid_credentials() -> AppError:
    return AppError(ErrorCode.INVALID_CREDENTIALS, 401, "Email or password is incorrect.")
```

문법 풀이

import 문법	뜻
`from app.core.security import …, verify_password`	16번의 비교 함수를 가져옵니다

식	뜻
`verify_password(password, DUMMY_PASSWORD_HASH)`	결과를 **쓰지 않고** 부르기만 합니다. 시간만 쓰게 하려는 것입니다
`raise invalid_credentials()`	함수가 만든 오류를 던집니다 (21번의 `unavailable_error`와 같은 방식)
`user.last_login_at = …` → `db.commit()`	회원 행의 마지막 로그인 시각을 바꿔 저장합니다
`.replace(tzinfo=None)`	DB 칸(`DATETIME`)에는 시간대 표시가 없으므로, UTC 시각에서 표시를 뗍니다 (19번과 같음)

4. 흐름

1. 이메일을 17번으로 정리한 뒤 회원을 찾습니다.
2. 회원이 없으면 가짜 해시로 한 번 비교하고(시간 맞추기) 401로 응답합니다.
3. 비밀번호가 틀리면 401로 응답합니다.
4. 탈퇴한 회원이면 401로 응답합니다 (같은 문구).
5. 정지된 회원이면 403으로 응답합니다.
6. 마지막 로그인 시각을 기록하고 회원을 돌려줍니다.

5. 왜 이렇게

선택	이유
틀린 이유를 알려 주지 않음	"이메일이 없음"과 "비밀번호가 틀림"을 구분해서 알려 주면, 남의 이메일이 가입되어 있는지 알아낼 수 있습니다 (FR-103 통일 문구)
없는 이메일도 가짜 해시로 비교	비교(약 0.2초)를 건너뛰면 없는 이메일에만 빨리 응답하게 됩니다. 그러면 응답 **시간**으로 가입 여부가 드러납니다. 이것을 막습니다 (16 §5에 적어 둔 일)
가짜 해시를 코드에 상수로 둠	서버가 켜질 때마다 0.2초씩 들여서 만들 필요가 없습니다. 비밀번호가 아니라 "아무것과도 맞지 않는 해시"라서 드러나도 문제가 없습니다. 비용 값이 12라서 진짜 해시와 걸리는 시간이 같습니다
탈퇴한 회원에게도 같은 401	탈퇴한 계정이 있었다는 사실조차 알려 주지 않습니다 (18번의 추적 방지와 같은 생각)
정지(403)는 **비밀번호가 맞은 뒤에만** 알림 (템플릿 순서 그대로)	틀린 비밀번호로는 "이 이메일은 정지되었다"는 사실을 알 수 없습니다. 비밀번호를 아는 본인에게만 정지 사실을 알려 줍니다
실패하면 `last_login_at`을 바꾸지 않음	성공한 로그인만 기록합니다

참고 (탈퇴 3일 뒤): 18번으로 비켜 둔 행은 이메일이 `deleted+…`로 바뀌어서 원래 이메일로는 찾을 수조차 없습니다. 또 P1-09 정리 작업이 비밀번호 해시를 `"!"`로 바꾸므로, 16번은 항상 `False`를 돌려줍니다.

6. 시험 (`tests/test_auth_service.py`에 7개를 추가해서 모두 44개, 실제 MySQL로 실행)

시험	확인한 것
성공	그 회원을 돌려주고, `last_login_at`이 지금(5초 안)으로 기록됩니다
대소문자와 공백	`" LOGIN@Example.com "`으로도 로그인됩니다 (17)
비밀번호가 틀림	401 "Email or password is incorrect."이고, `last_login_at`은 그대로입니다
없는 이메일	같은 401과 같은 문구이고, 걸린 시간이 0.1초 이상입니다 (가짜 해시 비교를 했다는 뜻입니다)
탈퇴한 회원, 맞는 비밀번호	401입니다 (틀린 비밀번호와 구분되지 않습니다)
정지된 회원	맞는 비밀번호면 403 "This account is disabled.", 틀린 비밀번호면 401입니다
가짜 해시	`""`, `"password"`, `"Abcdef1!"` 등 어떤 값과도 맞지 않습니다

진짜 해시로 로그인을 시험할 수 있도록, 시험 도우미 `add_user()`가 `password_hash`를 바꿔 넣을 수 있게 고쳤습니다.

전체 시험 209개가 통과했고, `ruff` 검사도 통과했습니다. 시험이 끝난 뒤 남은 행은 0개입니다.

Backend(FastAPI/Python) 30 — `get_active_user()`

항목	내용
파일	`studionx/backend/app/services/auth_service.py`
작업	P1-05 로그인: 재발급 전 회원 확인 (PRD 01 FR-105, FR-108)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	29 `authenticate()` · 31 리프레시 쿠키

1. 하는 일

회원 번호로 회원을 찾아서, **아직 활동 중(`ACTIVE`)**이면 돌려줍니다. 없거나, 정지되었거나, 탈퇴했으면 401로 응답합니다.

회원 번호에 해당하는 회원	결과
활동 중	`User`를 돌려줍니다
정지(`SUSPENDED`), 탈퇴(`DELETED`)	401 `UNAUTHORIZED` "Please sign in again."
없음	똑같이 401입니다

2. 언제 쓰나

쓰는 곳	하는 일
P1-05 `POST /auth/refresh` (33)	28번으로 리프레시 토큰에서 회원 번호를 꺼낸 뒤, 이 함수로 회원을 확인하고 새 Access Token을 줍니다

이 함수가 없으면 생기는 일

순서	일어나는 일
①	회원이 자동 로그인(3일 쿠키)으로 로그인합니다
②	그날 탈퇴합니다(P1-09). 브라우저에는 리프레시 쿠키가 아직 남아 있습니다
③	다음 날 사이트를 열면 `api.js`가 `/auth/refresh`를 부릅니다. 토큰은 진짜이고 3일 안이라서 28번은 통과합니다
④	**이 함수가 없다면** 탈퇴한 회원에게 새 Access Token이 나갑니다. 이 함수가 `DELETED`를 보고 401로 응답하기 때문에, 로그인 화면으로 가게 됩니다

정지(관리자가 `SUSPENDED`로 바꿈)도 마찬가지입니다. 바꾼 순간부터 재발급이 막히고, 남아 있는 Access Token(최대 30분)도 06번 `get_current_user()`가 막습니다.

06번 `get_current_user()`와 비교

	06 `get_current_user()`	30 `get_active_user()`
회원 번호를 얻는 곳	Access Token (05)	Refresh Token (28)
쓰는 곳	로그인이 필요한 모든 API	`/auth/refresh` 하나
검사	없거나 `ACTIVE`가 아니면 401	같습니다

3. 코드

import (파일 맨 위)

```
from datetime import UTC, datetime, timedelta

from sqlalchemy import select
from sqlalchemy.exc import IntegrityError
from sqlalchemy.orm import Session

from app.core.errors import AppError, ErrorCode
from app.core.security import check_password_rules, hash_password, verify_password
from app.models import User
from app.models.user import UserStatus
from app.schemas.auth import PRIVACY_VERSION, AvailabilityResponse, SignupRequest
```

`auth_service.py` 맨 위의 import입니다 (이번에 새로 추가한 것은 없습니다). 이 함수는 **`Session`, `AppError`, `ErrorCode`, `User`, `UserStatus`**를 사용합니다.

함수

```
def get_active_user(db: Session, user_id: int) -> User:
    user = db.get(User, user_id)
    if user is None or user.status != UserStatus.ACTIVE:
        raise AppError(ErrorCode.UNAUTHORIZED, 401, "Please sign in again.")
    return user
```

문법 풀이

import 문법	뜻
`from app.models.user import UserStatus`	08번의 회원 상태 목록입니다

식	뜻
`db.get(User, user_id)`	기본 키로 행 하나를 찾습니다. 없으면 `None`입니다 (06번과 같음)
`user is None or …`	앞이 참이면 뒤를 보지 않습니다. 그래서 `None.status` 오류가 나지 않습니다
`user.status != UserStatus.ACTIVE`	08번의 `StrEnum`끼리 비교합니다 (06번은 문자열 `"ACTIVE"`와 비교하지만 결과는 같습니다)

4. 흐름

1. 회원 번호로 회원을 찾습니다.
2. 없거나 활동 중이 아니면 401로 응답합니다.
3. 회원을 돌려줍니다.

5. 왜 이렇게

선택	이유
403이 아니라 `UNAUTHORIZED` 401	`api.js`는 재발급이 실패하면(401) 로그인 화면으로 보냅니다. 정지되거나 탈퇴한 회원도 "다시 로그인"으로 끝납니다. 정지 사실은 로그인(29)을 다시 시도했을 때 비밀번호가 맞으면 403으로 알려 줍니다
없는 회원, 정지, 탈퇴를 같은 응답으로 처리	05번, 06번과 같은 이유입니다. 이유를 알려 주지 않습니다
06번은 고치지 않음	템플릿은 "06번이 이 함수를 불러 써도 된다"고 했지만, 06번은 이미 검토가 끝난 코드이고 검사도 두 줄뿐입니다. 06번을 바꾸면 P1-03 문서와 시험도 다시 봐야 합니다

6. 시험 (`tests/test_auth_service.py`에 4개를 추가해서 모두 48개, 실제 MySQL로 실행)

시험	확인한 것
활동 회원	그 회원을 돌려줍니다
정지, 탈퇴 (2개)	401 `UNAUTHORIZED` "Please sign in again."
없는 회원 번호	401

전체 시험 213개가 통과했고, `ruff` 검사도 통과했습니다.

Backend(FastAPI/Python) 31 — `set_refresh_cookie()` · `clear_refresh_cookie()`

항목	내용
파일	`studionx/backend/app/api/v1/auth.py`
작업	P1-05 로그인: 리프레시 쿠키 (PRD 01 FR-105, FR-109)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	30 `get_active_user()` · 32 `POST /auth/login`

1. 하는 일

리프레시 토큰(27)을 브라우저 **쿠키**에 심고(`set`), 로그아웃할 때 지웁니다(`clear`). 두 함수 모두 응답 헤더 `Set-Cookie` 한 줄을 만듭니다.

호출	응답 헤더 `Set-Cookie`
`set_refresh_cookie(res, token, remember=True)`	`snx_refresh=eyJ…; HttpOnly; Max-Age=259200; Path=/api/v1/auth; SameSite=lax`
`set_refresh_cookie(res, token, remember=False)`	`snx_refresh=eyJ…; HttpOnly; Path=/api/v1/auth; SameSite=lax`
`clear_refresh_cookie(res)`	`snx_refresh=""; expires=(지금); HttpOnly; Max-Age=0; Path=/api/v1/auth; SameSite=lax`

쿠키 속성	값	뜻
`HttpOnly`	켬	**JS가 읽을 수 없습니다** (`document.cookie`에 보이지 않습니다). 그래서 스크립트 공격(XSS)이 훔쳐 가지 못합니다
`Path=/api/v1/auth`		`/api/v1/auth/…` 요청에만 실립니다. 다른 API나 페이지 요청에는 실리지 않습니다
`SameSite=lax`		다른 사이트가 몰래 보낸 `POST` 요청에는 실리지 않습니다 (CSRF 대비)
`Max-Age`	259200초(3일) 또는 없음	없으면 **세션 쿠키**가 되어 브라우저를 닫을 때 사라집니다
`Secure`	배포 환경(`APP_ENV=prod`)에서만 켬	HTTPS로만 보냅니다. 개발 환경(`http://localhost`)에서 켜면 쿠키가 아예 실리지 않아서 꺼 둡니다

2. 언제 쓰나

쓰는 곳	함수	화면
32 `POST /auth/login`	`set_refresh_cookie(…, remember=data.remember)`	로그인 모달의 **Keep me signed in** 체크 여부
34 `POST /auth/logout`	`clear_refresh_cookie()`	헤더의 **Sign Out** (JS 08)

Keep me signed in	브라우저를 닫았다가 다시 열면
체크함	3일 동안 쿠키가 남아 있어서 `api.js`가 재발급을 받고, 로그인된 상태가 유지됩니다

Keep me signed in	브라우저를 닫았다가 다시 열면
체크하지 않음	쿠키가 사라져서 재발급에 실패하고, 로그인 화면으로 갑니다

3. 코드

import (파일 맨 위)

```
from typing import Annotated

from fastapi import APIRouter, Cookie, Query, Response, status
from pydantic import EmailStr

from app.api.deps import Db
from app.core.config import get_settings
from app.schemas.auth import (
    AvailabilityResponse,
    LoginRequest,
    Nickname,
    SignupRequest,
    SignupResponse,
    TokenResponse,
)
from app.schemas.common import ErrorResponse
from app.services import auth_service
```

`auth.py` 맨 위의 import입니다. **`from app.core.config import get_settings`를 이번에 추가했습니다.** 이 함수들은 `Response`와 `get_settings`를 사용합니다.

함수

```
REFRESH_COOKIE = "snx_refresh"
REFRESH_COOKIE_PATH = "/api/v1/auth"


def set_refresh_cookie(response: Response, token: str, remember: bool) -> None:
    settings = get_settings()
    if remember:
        max_age = settings.jwt_refresh_days * 24 * 60 * 60
    else:
        max_age = None
    response.set_cookie(
        key=REFRESH_COOKIE,
        value=token,
        path=REFRESH_COOKIE_PATH,
        httponly=True,
        samesite="lax",
        secure=settings.app_env == "prod",
        max_age=max_age,
    )


def clear_refresh_cookie(response: Response) -> None:
    response.delete_cookie(
        REFRESH_COOKIE,
```

```
        path=REFRESH_COOKIE_PATH,
        httponly=True,
        samesite="lax",
        secure=get_settings().app_env == "prod",
    )
```

문법 풀이

import 문법	뜻
`from app.core.config import get_settings`	`.env`를 읽어 만든 설정입니다 (05번과 같음)

식	뜻
`response: Response`	라우터가 받은 응답 객체입니다. 여기에 쿠키를 붙이면 응답 헤더로 나갑니다
`response.set_cookie(key=…, value=…, …)`	`Set-Cookie` 헤더 한 줄을 만듭니다
`max_age=None`	`Max-Age`를 아예 쓰지 않습니다. 그러면 세션 쿠키가 됩니다
`settings.app_env == "prod"`	배포 환경이면 `True`입니다
`response.delete_cookie(…)`	값이 비어 있고 `Max-Age=0`인 같은 이름의 쿠키를 보내서, 브라우저가 쿠키를 지우게 합니다
`-> None`	돌려주는 값이 없습니다. `response`를 바꾸는 것이 이 함수의 일입니다

4. 흐름

심기: 수명을 정하고(체크했으면 3일, 아니면 없음), 속성을 붙여 `Set-Cookie`를 만듭니다.

지우기: 같은 이름과 같은 경로로 "지금 만료되는" 쿠키를 보냅니다.

5. 왜 이렇게

선택	이유
응답 본문이 아니라 쿠키에 담음	본문으로 주면 JS가 어딘가에 저장해야 합니다. `HttpOnly` 쿠키는 JS가 손댈 수 없고, 브라우저가 알아서 보냅니다
경로를 `/api/v1/auth`로 좁힘	3일짜리 토큰이 모든 요청에 실려 다니지 않습니다. 재발급과 로그아웃할 때만 서버로 갑니다
체크하지 않으면 세션 쿠키	공용 컴퓨터에서 브라우저를 닫으면 로그인이 풀립니다 (FR-109)
`Secure`는 배포 환경에서만 켬	개발 서버는 `http://`라서 `Secure` 쿠키가 실리지 않습니다. 배포 환경(HTTPS)에서는 반드시 켭니다
지울 때도 같은 속성을 씀	브라우저는 이름과 경로가 같아야 같은 쿠키로 보고 지웁니다. 템플릿은 경로만 맞췄지만, 속성까지 같게 해 두면 브라우저마다 다르게 처리할 걱정이 없습니다

6. 시험 (`tests/test_auth_cookies.py` 새 파일, 5개)

시험	확인한 것
체크함	`snx_refresh=TOKEN`, `Max-Age=259200`, `Path=/api/v1/auth`, `HttpOnly`, `SameSite=lax`
체크하지 않음	`Max-Age`와 `expires`가 없고(세션 쿠키), `HttpOnly`가 있습니다
`Secure` (2개)	설정을 `dev`로 바꾸면 없고, `prod`로 바꾸면 있습니다
지우기	값이 비어 있고, `Max-Age=0`이며, 경로가 같습니다

전체 시험 218개가 통과했고, `ruff` 검사도 통과했습니다.

Backend(FastAPI/Python) 32 — 라우터 `POST /api/v1/auth/login`

항목	내용
파일	`studionx/backend/app/api/v1/auth.py` (함수 이름 `login`)
작업	P1-05 로그인: 로그인 API (PRD 01 FR-103, FR-105, FR-109)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	31 리프레시 쿠키 · 33 `POST /auth/refresh`

1. 하는 일

이메일과 비밀번호를 확인하고, **토큰 두 개**를 서로 다른 경로로 나눠서 줍니다.

```
POST /api/v1/auth/login
{"email": "route.user@example.com", "password": "Abcdef1!", "remember": true}

→ 200
Set-Cookie: snx_refresh=eyJ…(3일); HttpOnly; Max-Age=259200; Path=/api/v1/auth; SameSite=lax
{"access_token": "eyJ…(30분)", "token_type": "bearer", "expires_in": 1800}
```

토큰	전달되는 곳	만드는 함수
Access (30분)	응답 **본문**으로 가서 JS 메모리에 보관됩니다	26
Refresh (3일)	응답 **쿠키**(`HttpOnly`)로 가며, JS는 볼 수 없습니다	27, 31

경우	응답
맞음	200과 위의 내용
비밀번호 틀림, 없는 이메일, 탈퇴 회원	401 `INVALID_CREDENTIALS` "Email or password is incorrect."이고, **쿠키는 없습니다**
정지된 회원 (비밀번호는 맞음)	403 `ACCOUNT_DISABLED`
칸이 빠짐	422

2. 언제 쓰나

home.html 로그인 모달의 **Sign In** 버튼을 누르면 `home.js`와 `auth-ui.js`(P1-07)가 이 API를 부릅니다.

순서	화면 또는 서버	일어나는 일
①	로그인 모달	이메일, 비밀번호, **Keep me signed in**을 입력합니다
②	P1-07 JS	`api.post('/auth/login', {email, password, remember}, {auth: false})`를 보냅니다
③	**이 라우터**	29번으로 확인하고, 27번과 31번으로 쿠키를 만든 뒤, 26번으로 본문을 만듭니다
④	P1-07 JS	`setAccessToken(access_token)`을 부르고(JS 01), 헤더를 My Page · My Projects로 바꾼 뒤 `next` 페이지로 이동합니다 (JS 05)

3. 코드

import (파일 맨 위)

```
from typing import Annotated

from fastapi import APIRouter, Cookie, Query, Response, status
from pydantic import EmailStr

from app.api.deps import Db
from app.core.config import get_settings
from app.core.security import create_access_token, create_refresh_token
from app.schemas.auth import (
    AvailabilityResponse,
    LoginRequest,
    Nickname,
    SignupRequest,
    SignupResponse,
    TokenResponse,
)
from app.schemas.common import ErrorResponse
from app.services import auth_service
```

`auth.py` 맨 위의 import입니다. **`from app.core.security import create_access_token, create_refresh_token`을 이번에 추가했습니다.** 이 함수는 `Response`, `Db`, `get_settings`, 두 토큰 함수, `LoginRequest`, `TokenResponse`, `auth_service`를 사용합니다.

함수

```
@router.post(
    "/login",
    response_model=TokenResponse,
    responses={401: {"model": ErrorResponse}, 403: {"model": ErrorResponse}},
)
def login(data: LoginRequest, response: Response, db: Db) -> TokenResponse:
    user = auth_service.authenticate(db, data.email, data.password)
    set_refresh_cookie(response, create_refresh_token(user.user_id), data.remember)
    return TokenResponse(
        access_token=create_access_token(user.user_id),
        expires_in=get_settings().jwt_access_minutes * 60,
    )
```

`@router.post(…)`와 함수 머리는 템플릿에 있었고, 몸통 세 줄을 채웠습니다.

문법 풀이

import 문법	뜻
`from app.core.security import A, B`	26번과 27번의 토큰 함수 두 개를 가져옵니다

식	뜻
`response: Response`	FastAPI가 넣어 주는 응답 객체입니다. 여기에 쿠키를 붙이면(31) 최종 응답 헤더에 실립니다

식	뜻
`data: LoginRequest`	본문 `{email, password, remember}` 입니다 (`remember`가 없으면 `False`)
`TokenResponse(access_token=…, expires_in=1800)`	응답 본문입니다. `token_type`은 기본값 `"bearer"` 입니다

4. 흐름

1. 29번 `authenticate()`로 확인합니다. 틀리면 여기서 401이나 403으로 끝납니다 (쿠키도 토큰도 만들지 않습니다).
2. 27번으로 리프레시 토큰을 만들고, 31번으로 쿠키에 담습니다 (`remember`에 따라 3일 쿠키 또는 세션 쿠키).
3. 26번으로 Access Token을 만들어 본문에 담고 200으로 응답합니다.

5. 왜 이렇게

선택	이유
확인을 맨 먼저 함	실패하면 토큰도 쿠키도 만들지 않습니다. 시험에서 실패 응답에 `Set-Cookie`가 없다는 것을 확인했습니다
Access는 본문으로, Refresh는 쿠키로	27 §5에서 설명했듯이, 짧은 토큰은 JS가 쓰고 긴 토큰은 JS가 볼 수 없게 숨깁니다
`expires_in`을 함께 줌	화면이 "몇 초 뒤에 만료되는지" 알 수 있습니다. 지금 `api.js`는 401을 받으면 재발급하는 방식이라 쓰지 않지만, 만료 전에 미리 재발급하는 방식으로 바꿀 때 쓸 수 있습니다
이메일 형식 검사 (`EmailStr`)를 하지 않음	로그인 칸에 형식이 틀린 이메일을 넣어도 422가 아니라 401 "Email or password is incorrect." 하나로 답합니다. 무엇이 틀렸는지 알려 주지 않는 원칙(29)과 같습니다

6. 시험 (`tests/test_auth_routes.py`에 7개를 추가해서 모두 34개, 실제 앱과 MySQL로 실행)

가입(22)으로 회원을 만든 뒤 로그인했습니다.

시험	확인한 것
200 본문	칸이 `access_token`, `token_type`, `expires_in` 세 개뿐이고(리프레시 토큰은 없음), `bearer`와 1800이며, 05번으로 읽으면 그 회원 번호가 나옵니다
쿠키 (체크함, 체크하지 않음)	`snx_refresh=…; HttpOnly; Path=/api/v1/auth`이고, 체크했으면 `Max-Age=259200`, 체크하지 않았으면 없습니다. 쿠키 값을 28번으로 읽으면 그 회원 번호가 나옵니다
마지막 로그인	`last_login_at`이 기록됩니다
비밀번호 틀림, 없는 이메일	둘 다 401이고 본문이 같으며, `Set-Cookie`가 없습니다
비밀번호 칸이 빠짐	422

전체 시험 225개가 통과했고, `ruff` 검사도 통과했습니다. 시험이 끝난 뒤 남은 행은 0개입니다.

Backend(FastAPI/Python) 33 — 라우터 `POST /api/v1/auth/refresh`

항목	내용
파일	`studionx/backend/app/api/v1/auth.py` (함수 이름 `refresh`)
작업	P1-05 로그인: Access Token 재발급 (PRD 01 FR-105, FR-108)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	32 `POST /auth/login` · 34 `POST /auth/logout`

1. 하는 일

브라우저에 남아 있는 **리프레시 쿠키**로 새 Access Token(30분)을 줍니다. 요청 본문은 없으며, 쿠키가 자동으로 실려 옵니다.

```
POST /api/v1/auth/refresh
Cookie: snx_refresh=eyJ…

→ 200 {"access_token": "eyJ…(새 30분)", "token_type": "bearer", "expires_in": 1800}
```

쿠키	응답	화면 (`api.js`)
정상 (3일 안, 활동 회원)	200과 새 Access Token	메모리에 저장하고, 하던 요청을 계속합니다
없음 (로그인하지 않은 방문자)	401 "Please sign in."	로그인하지 않은 상태로 처리합니다 (정상적인 흐름)
만료, 위조, Access Token, 엉터리 값	401 "Please sign in again."	로그인 화면으로 보냅니다
진짜지만 회원이 탈퇴했거나 정지됨	401 "Please sign in again."	로그인 화면으로 보냅니다

2. 언제 쓰나

P1-02의 `refreshAccessToken()`이 부릅니다. **이 라우터가 생기면서 P1-02의 재발급이 실제로 동작하게 되었습니다.**

상황	부르는 쪽
페이지를 새로 열었을 때 (메모리의 토큰이 비어 있음)	`apiFetch`가 요청 전에 먼저 부릅니다 (JS 02)
30분이 지나 API가 401로 응답할 때	`apiFetch`가 재발급을 받은 뒤 같은 요청을 한 번 다시 보냅니다
헤더 버튼을 그릴 때, 로그인 전용 페이지를 열 때	`isLoggedIn()`과 `requireLogin()` (JS 06, 07)

ai-note 34(P1-02, 03 실제 사이트 시험)에서 `/auth/refresh → 500`이었던 결과가 이제 200과 401로 바뀝니다.

3. 코드

import (파일 맨 위)

```
from typing import Annotated

from fastapi import APIRouter, Cookie, Query, Response, status
```

```
from pydantic import EmailStr

from app.api.deps import Db
from app.core.config import get_settings
from app.core.errors import AppError, ErrorCode
from app.core.security import create_access_token, create_refresh_token, decode_refresh_token
from app.schemas.auth import (…, TokenResponse)
from app.schemas.common import ErrorResponse
from app.services import auth_service
```

`auth.py` 맨 위의 import입니다. **`AppError`, `ErrorCode`, `decode_refresh_token`을 이번에 추가했습니다.** 이 함수는 `Annotated`, `Cookie`, `Response`, `Db`, `get_settings`, `AppError`, `ErrorCode`, `create_access_token`, `decode_refresh_token`, `TokenResponse`, `auth_service`를 사용합니다.

함수

```
@router.post(
    "/refresh",
    response_model=TokenResponse,
    responses={401: {"model": ErrorResponse}},
)
def refresh(
    response: Response,
    db: Db,
    refresh_token: Annotated[str | None, Cookie(alias=REFRESH_COOKIE)] = None,
) -> TokenResponse:
    if refresh_token is None:
        raise AppError(ErrorCode.UNAUTHORIZED, 401, "Please sign in.")
    user_id = decode_refresh_token(refresh_token)
    user = auth_service.get_active_user(db, user_id)
    return TokenResponse(
        access_token=create_access_token(user.user_id),
        expires_in=get_settings().jwt_access_minutes * 60,
    )
```

함수 머리는 템플릿에 있었고, 몸통을 채웠습니다.

문법 풀이

import 문법	뜻
`from fastapi import …, Cookie, …`	요청의 쿠키에서 값을 꺼낸다는 표시입니다 (템플릿에 이미 있었습니다)

식	뜻
`Annotated[str | None, Cookie(alias="snx_refresh")]`	쿠키 `snx_refresh`의 값을 `refresh_token` 인자로 받습니다. 파이썬 이름과 쿠키 이름이 달라서 `alias`로 연결합니다
`= None`	쿠키가 없으면 `None`입니다 (오류가 아닙니다)
`response: Response`	지금은 쓰지 않습니다. 나중에 리프레시 토큰을 새로 갈아 끼울 때(아래 §5) 쿠키를 심는 데 씁니다

4. 흐름

1. 쿠키가 없으면 401 "Please sign in."으로 응답합니다.

2. 28번으로 토큰을 검사해서 회원 번호를 얻습니다 (실패하면 401 "Please sign in again.").
3. 30번으로 회원이 아직 활동 중인지 확인합니다 (아니면 401).
4. 26번으로 새 Access Token을 만들어 200으로 응답합니다.

5. 왜 이렇게

선택	이유
쿠키가 없을 때와 토큰이 이상할 때의 문구를 다르게 함	쿠키가 없는 것은 "처음부터 로그인하지 않았다"는 뜻이라 정상입니다. 쿠키가 있는데 안 되는 것은 "로그인이 풀렸다"는 뜻입니다. 코드는 둘 다 `UNAUTHORIZED`이므로 `api.js`의 처리는 같습니다 (06번과 같은 구분)
토큰 검사(28) 다음에 회원 확인(30)	토큰은 진짜여도 그사이 회원이 탈퇴하거나 정지되었을 수 있습니다 (30 §2)
리프레시 토큰을 갈아 끼우지 않음 (템플릿의 선택 사항)	갈아 끼우면 재발급할 때마다 3일이 다시 늘어나서 사실상 영원히 로그인된 상태가 됩니다. 또 체크 여부(`remember`)를 토큰에 따로 담아야 합니다. 지금은 **로그인하고 3일이 지나면 다시 로그인**하는 단순한 방식으로 둡니다
실패해도 쿠키를 지우지 않음	오류 응답은 01번 처리기가 새로 만들기 때문에, 여기서 붙인 쿠키가 실리지 않습니다. 쿠키는 3일이 지나면 브라우저가 지우고, 로그아웃(34)할 때도 지웁니다

6. 시험 (`tests/test_auth_routes.py`에 6개를 추가해서 모두 40개, 실제 앱과 MySQL로 실행)

시험 도구(`TestClient`)는 브라우저처럼 로그인 응답의 쿠키를 기억했다가 다음 요청에 실어 보냅니다.

시험	확인한 것
로그인 후 재발급 (체크함, 체크하지 않음)	200, `bearer`, 1800이고, 새 토큰을 05번으로 읽으면 그 회원 번호가 나옵니다
쿠키 없음	401 "Please sign in."
엉터리 쿠키, Access Token을 쿠키에 넣음	401 `UNAUTHORIZED` "Please sign in again."
로그인 뒤 탈퇴	쿠키가 남아 있어도 401입니다 (30번이 막습니다)

전체 시험 231개가 통과했고, `ruff` 검사도 통과했습니다. 시험이 끝난 뒤 남은 행은 0개입니다.

Backend(FastAPI/Python) 34 — 라우터 `POST /api/v1/auth/logout`

항목	내용
파일	`studionx/backend/app/api/v1/auth.py` (함수 이름 `logout`)
작업	P1-05 로그인: 로그아웃 (PRD 01 FR-104, P1-05 마지막 함수)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	33 `POST /auth/refresh` · (P1-05 끝) · 실제 사이트 시험 ai-note 37

1. 하는 일

리프레시 쿠키를 **지우라는 응답**을 보냅니다. 본문 없이 204로 응답합니다.

```
POST /api/v1/auth/logout

→ 204 No Content
Set-Cookie: snx_refresh=""; expires=(지금); HttpOnly; Max-Age=0; Path=/api/v1/auth; SameSite=lax
```

토큰	로그아웃할 때 지우는 쪽
Refresh (쿠키)	**이 라우터**입니다. JS는 `HttpOnly` 쿠키를 지울 수 없어서 서버가 지워야 합니다
Access (JS 메모리)	화면 `logout()`의 `clearAccessToken()`입니다 (JS 08)

2. 언제 쓰나

헤더의 **Sign Out** 버튼을 누르면 JS 08 `logout()`이 이 API를 부릅니다.

순서	화면 또는 서버	일어나는 일
①	`logout()`	`api.post('/auth/logout', undefined, { auth: false })`를 보냅니다
②	**이 라우터**	204로 응답하면서 쿠키를 지웁니다
③	`logout()`	`clearAccessToken()`을 부르고 `home.html`로 이동합니다
④	다음에 사이트를 열면	쿠키가 없어서 재발급(33)이 401이 되고, 로그인하지 않은 상태가 됩니다

ai-note 34(P1-02, 03 실제 사이트 시험)에서 `/auth/logout → 500`이었던 결과가 이제 204가 됩니다. (그때도 `logout()`은 실패를 무시하고 홈으로 갔지만, 쿠키는 남아 있었습니다.)

3. 코드

import (파일 맨 위: 32번, 33번과 같고, 이번에 새로 추가한 것은 없습니다)

```
from fastapi import APIRouter, Cookie, Query, Response, status
…
```

이 함수는 **`Response`와 `status`**를 쓰고, 쿠키를 지우는 일은 31번 `clear_refresh_cookie()`에 맡깁니다.

함수

```
@router.post("/logout", status_code=status.HTTP_204_NO_CONTENT)
def logout(response: Response) -> None:
    clear_refresh_cookie(response)
```

문법 풀이

import 문법	뜻
`from fastapi import status`	상태 번호에 이름을 붙여 둔 모음입니다 (`HTTP_204_NO_CONTENT` = 204)

식	뜻
`status_code=204`	성공했지만 **본문이 없다**는 뜻입니다
`-> None`	돌려줄 본문이 없습니다
`response: Response`	여기에 붙인 쿠키 삭제 헤더가 204 응답에 실립니다
`db`와 `user` 인자가 없음	로그인 확인도 하지 않고, DB도 쓰지 않습니다

4. 흐름

1. 31번으로 "지금 만료되는" 쿠키를 응답에 붙입니다.
2. 204로 응답합니다.

5. 왜 이렇게

선택	이유
로그인 확인 없이 받음 (템플릿 권장)	Access Token이 이미 만료된 상태에서 Sign Out을 눌러도 쿠키는 지워져야 합니다. 하는 일이 "쿠키 지우기"뿐이라서 누가 불러도 위험하지 않습니다
서버에 아무것도 기록하지 않음	토큰을 서버에 저장하지 않는 방식(JWT)이기 때문입니다. 남아 있는 Access Token은 길어야 30분 뒤에 스스로 만료됩니다
204로 응답	돌려줄 내용이 없다는 표준 응답입니다. `api.js`는 204를 `null`로 처리합니다 (JS 02)

참고 (쿠키를 훔쳐 간 경우): 로그아웃은 **이 브라우저**의 쿠키만 지웁니다. 이미 복사해 간 리프레시 토큰은 3일 동안 유효합니다. 이것을 막으려면 서버에 "무효 토큰 목록"을 두거나, 비밀번호를 바꿀 때 모든 토큰을 끊는 기능이 필요합니다. Phase 1 범위 밖이라서 나중에 검토합니다.

6. 시험 (`tests/test_auth_routes.py`에 3개를 추가해서 모두 43개, 실제 앱과 MySQL로 실행)

시험	확인한 것
로그인 → 로그아웃 → 재발급	로그아웃 전 재발급은 200이고, 로그아웃은 204에 본문이 없으며 `snx_refresh=""; Max-Age=0; Path=/api/v1/auth` 헤더가 붙습니다. 로그아웃 뒤 재발급은 401입니다
로그인하지 않은 상태	204
엉터리 `Authorization` 헤더	204입니다 (로그인 확인을 하지 않습니다)

전체 시험 234개가 통과했고, `ruff` 검사도 통과했습니다. 인증 관련 파일(`auth.py`, `security.py`, `auth_service.py`)에 남은 `NotImplementedError`는 0개입니다.